DocMind 통합보고서

구축 과정 · 성능과 리소스 검증 · 엔진 인수인계

AVDC CPU/OpenVINO · OCR 미사용 · 2026-11-10

최신 실자료

154개 / 100.74MiB / 정리 ACK까지 185.18초

같은 입력 GPU 비교

24개: GPU 39.19초 / AVDC OpenVINO 61.70초

최신 서버 실제 사용률

154개 시험 평균 CPU 25% / 관측 최대 83%

인수 개발자용

자산 수령 → 설정 → 연결 → 실행 → 합격 확인

이전 성능 PDF와 보완한 인수인계서를 한 문서로 통합했습니다. 현재 적용 기준과 과거 시험·미검증 범위를 분리했습니다.

목차

읽는 순서와 보고 범위 3

1. 최신 결과 — 실제 154개를 약 3분 5초에 처리 4

2. 핵심 비교 — 같은 입력에서 GPU와 CPU 5

3. 구축 구조와 현재 리소스 배분 6

4. 실제 CPU·메모리·그룹웨어 부하 7

5. 복호화부터 완료까지 단계별 시간 8

6. 최적화 이력 — 현재 설정과 과거 조건을 분리 9

7. OpenVINO·청킹 품질과 확인하지 못한 원인 11

8. 엔진 전달 범위·코드·배포 위치 13

9. 수령할 실행 자산과 전달·복원 방법 15

10. 실행 설정 작성과 확인 18

11. Windows 워커와 엔진 API 연동 20

12. 개발자 실행 순서와 합격 확인 24

13. 진단·검증 범위·남은 업무 28

14. 보관·용량·대량 처리 추정과 근거 31

부록 A. 실제 확인한 자산 식별자 33

부록 B. 통합에 사용한 PDF 원본 식별자 36

읽는 순서와 보고 범위

이 문서 하나에 최신 적용 상태, GPU 대비 성능, 최적화 이력, 실행 자산, 설정, API 연동, 실행 순서와 합격 확인 방법을 담았다. 이전 PDF 9종은 참고 이력으로 보관하며, 현재 판단에는 이 통합본을 우선한다.

관리·보고 담당자는 1~7장, 인수 개발자는 8~14장을 읽는다. 본문 명령은 실행할 위치와 전제를 확인한 후 사용하는 절차이며 이번 통합 작업에서 재색인·재배포·네트워크 변경은 하지 않았다. PDF의 긴 명령어는 화면 폭에 맞춰 접히므로 복사 실행할 때는 같은 통합본 Markdown의 코드 블록을 사용한다.

기록 시점의 확인과 현재 실시간 상태를 구분한다. 코드/인수 도구의 통합 직전 기준은 0dc194ad2c8f686521d096f5d3adbea8b9cbd915이다. 실제 이미지·모델·마운트 소스는 부록의 검증 목록과 대조한다. 문서 커밋이 바뀌어도 서버가 자동 배포되는 것은 아니다.

개발자의 전달 범위는 엔진이다. 운영 체계·네트워크·백업·사용자 인증 연동의 완료 여부는 엔진 구현과 구분해 남긴다. 새 서버 전체 설치 ZIP 또는 상용 복호화 모듈·실제 키는 이 PDF에 포함하지 않는다.

1. 최신 결과 — 실제 154개를 약 3분 5초에 처리

2026-11-10 확인. 기존 AVDC 문서 등록 689건과 검색 항목 45,636개를 백업 후 삭제하고 0건에서 시작했습니다. Windows의 암호화 원본은 보존했습니다. 입력 100.74 MiB는 십진수 단위로 약 105.64 MB입니다.

항목	이전 11월 6일	이번 11월 10일
같은 입력	154개 / 100.74 MiB	154개 / 100.74 MiB
등록 → 마지막 색인 활성화	205.08초	184.64초
등록 → 임시 파일 정리 ACK	204.98초	185.18초 (3분 5초)
성공 / OCR 필요 제외	152개 / 2개	152개 / 2개
완료 페이지 / 활성 청크	456쪽 / 2,135개	456쪽 / 1,596개
서버 CPU 평균 / 관측 최대	23.59% / 63%	25.00% / 83%

전체 색인 시간은 약 10% 짧아졌습니다

같은 154개 원본의 크기·수정 시각·SHA256을 확인했습니다. 파일별 성공·제외 결과도 이전과 같습니다. 새 청킹 규칙과 빈 색인 조건이 함께 적용됐으므로, 20.44초 단축을 청킹 하나의 효과로 단정하지 않습니다.

원본과 정리 상태까지 확인했습니다

현재 문서 등록은 154개, 활성 문서는 152개, 검색 청크는 1,596개입니다. 원본 해시는 모두 유지됐고 Windows·API·파서의 임시 평문 파일은 0개였습니다. OCR이 필요한 2개는 설정대로 제외됐으며 새 오류는 없었습니다.

등록부터의 시간에는 복호화·전송·대기·파싱·청킹·임베딩·색인 활성화가 포함됩니다. 설치·백업·색인 삭제·평상시 부하 측정과 사후 검증 시간은 제외합니다. 각 조건은 단회 실측입니다.

2. 핵심 비교 — 같은 입력에서 GPU와 CPU

이 페이지는 11월 6일까지의 기존 기록입니다. 24개·41.37 MiB 중 22개 성공, OCR 필요 2개 제외. 완료 242쪽·1,359청크로 같은 입력과 결과를 맞춰 비교했습니다.

GPU PC 39.191초 / PyTorch CPU 70.159초 / OpenVINO CPU 61.697초.

관측 항목	PyTorch CPU	OpenVINO CPU
전체 색인 시간	70.159초	61.697초
서버 CPU 평균 / 관측 최대	28.64% / 78%	34.20% / 87%
전체 컨테이너 최고 메모리	41.92 GiB	31.78 GiB
그룹웨어 HTTP 평균 / p95	19.93 / 35.99 ms	14.63 / 25.81 ms

무엇을 뜻하는 비교인가

GPU PC 39.191초 대비 OpenVINO CPU는 22.506초 더 걸렸습니다. GPU와 AVDC는 장비·파서·동시성이 다른 전체 시스템 비교입니다. 순수 CPU/GPU 칩 성능 배수로 해석하면 안 됩니다.

OpenVINO의 주 비교는 같은 상세 계측의 PyTorch 70.159초이며 12.06% 단축됐습니다. 66.684초는 앞선 비계측 기록이어서 같은 기준으로 섞지 않습니다. GPU 시험의 조정기 종료 표시 문제는 DB·ES·원본 해시·정리 ACK로 별도 검증한 이력이 있습니다.

이번 154개와 GPU 기록은 직접 나누지 않습니다

154개를 GPU로 같은 조건에 재측정한 값은 없습니다. 오늘의 184.64초와 과거 24개 GPU의 39.191초를 나눠 배수로 보고하지 않습니다. 오늘 시간당 31,118청크는 청킹 변경으로 청크 수가 줄어든 환산값이며, 이전 37,478청크/시간과 단독으로 속도를 판정하지 않습니다.

시간당 청크 수 환산: GPU 124,835개 / PyTorch CPU 69,733개 / OpenVINO CPU 79,297개. 같은 1,359청크를 각 단회 전체 시간으로 나눈 값이며 한 시간 연속 실측이 아니다.



3. 구축 구조와 현재 리소스 배분

단계	처리 위치와 역할
1. 원본 확인·복호화	Windows 워커가 등록한 원본의 안정성과 해시를 확인하고 uEncryptor2로 임시 복호화
2. 읽기·청킹	Hyper-V Ubuntu VM의 Docker 파서가 본문을 추출하고 DocMind가 청크 생성
3. 의미 계산·저장	OpenVINO CPU 임베딩 → Elasticsearch 저장 → 최신 청크 세트 활성화
4. 정리·완료	VM과 Windows 임시 평문을 삭제하고 정리 ACK 확인

허용량과 실제 사용량을 구분합니다

구성	최대 허용량 / 현재 설정
AVDC 물리 서버	CPU 칩 2개 / 물리 코어 48개 / 논리 프로세서 96개 / RAM 127.6 GiB
DocMind VM	96 vCPU · 상한 100% · RAM 64 GiB
임베딩	22작업자 × 4스레드 / CPU 시간 88개 논리 프로세서분 한도 / RAM 52 GiB 한도
파서	2작업자 / 합산 CPU 시간 4개분 한도 / RAM 1 GiB / OCR 꺼짐
동시 처리	문서 12슬롯 / 임베딩 요청 최대 24개 / 입력 문서 순서 유지
모델·내부 배치	GTE multilingual base · 768차원 · OpenVINO FP32 / 128토큰 이하 최대 16개, 긴 입력 1개

88개분은 CPU 칩 88개라는 뜻이 아닙니다. 이번 임베딩의 실제 평균 CPU 사용 시간은 약 18.28개 논리 프로세서분이었습니다. 전체 서버 평균 25%와는 측정 범위가 다릅니다.

VM 내부에서 임베딩은 논리 번호 0-47·56-95, 파서·DB·API 등은 48-55를 사용합니다. 이 분리는 그룹웨어용 물리 코어를 독점 예약한 것이 아닙니다. 로컬 PC GPU 구성과 NAT는 변경하지 않았습니다.

Windows	원본 확인 → 공유 게이트 → 임시 복호화
Ubuntu VM	파싱 → 청킹 → OpenVINO 임베딩
저장·활성화	Elasticsearch + MySQL → 최신 청크 세트
완료	Windows·VM 평문 정리 → 호스트 ACK

4. 실제 CPU·메모리·그룹웨어 부하

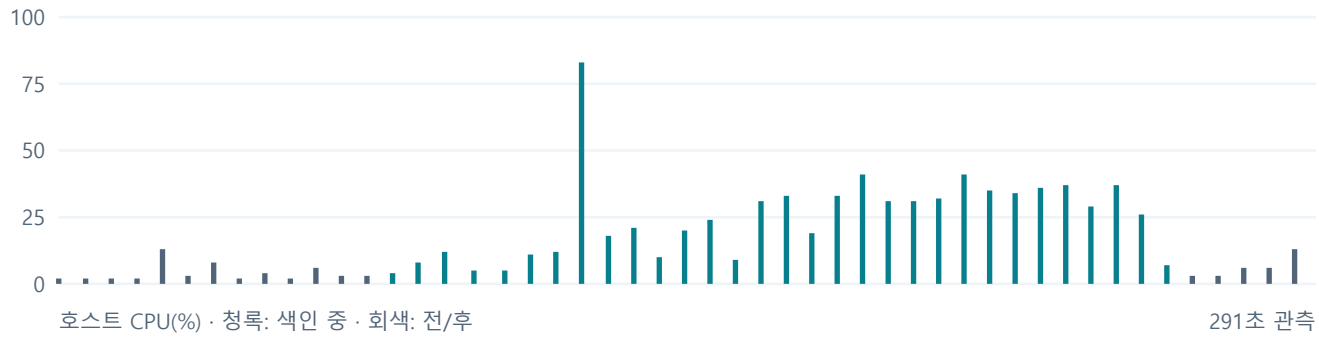
CPU 시계열은 PDF와 집계 JSON에 수록했다.

동일 실행의 관측 지표	색인 전	색인 중
호스트 표본 수 / HTTP 200	13 / 13	31 / 31
서버 전체 CPU 평균 / 최대	4.00% / 13%	25.00% / 83%
그룹웨어 VM CPU 평균 / 최대	0.08% / 1%	0% / 0%
그룹웨어 HTTP 평균	20.67 ms	22.18 ms
그룹웨어 HTTP p95 / 최대	31.90 / 31.90 ms	68.75 / 130.46 ms
디스크 대기열 평균 / 최대	0.31 / 1	0.16 / 1
디스크 읽기 / 쓰기 평균	0.106 / 0.071 ms	0.109 / 0.051 ms

HTTP 실패는 없었지만 평균 응답은 1.51ms 늘었고 p95·최대 지연도 증가했습니다. 그룹웨어 CPU 0%는 정수 단위 표본 값이므로 실제 사용량이 완전히 0이라는 뜻은 아닙니다.

색인 중 호스트 가용 RAM 최저 18.63 GiB. DocMind 컨테이너 합계 메모리 관측 최대 37.17 GiB, 임베딩 33.74 GiB, 파서 0.86 GiB, API 1.07 GiB였습니다. 개별 최대의 시점은 서로 다를 수 있습니다.

호스트 약 6초·컨테이너 약 1초 간격 측정입니다. 최대값은 표본에서 관측한 값입니다. HTTP는 그룹웨어 시작 페이지이며 로그인 후 결재·첨부·다중 사용자 부하를 재현한 시험은 아닙니다.



5. 복호화부터 완료까지 단계별 시간

같은 154개 입력. 아래 서버 단계는 성공한 152개 문서의 평균이며, 실패·제외 2개는 포함하지 않습니다. 문서와 단계가 병렬로 실행되므로 합계는 실제 경과 시간과 다릅니다.

서버 단계 (초/문서)	이전 평균	이번 평균
파서 API 왕복	0.642	0.665
추출 결과 정규화	0.026	0.027
청크 생성	0.063	0.050
임베딩 / 대기 포함	4.866	5.200
검색 색인 저장	0.190	0.164
최신 세트 활성화	0.181	0.156
VM 임시 파일 정리	0.092	0.056

이번 실행의 Windows 단계

단계	표본	평균 (초)	최대 (초)
원본 검증	154	0.224	0.639
복호화 차례 대기	154	3.274	18.526
복호화 호출·결과 확인	154	0.633	2.072
복호화 후 검증	154	0.140	0.337
Windows 평문 삭제	154	0.033	0.093
정리 ACK	154	0.197	1.074

파서 API 왕복은 파서 내부 실행 시간과 다릅니다. 임베딩 단계에도 서비스 대기가 들어갈 수 있습니다. Windows 복호화 호출에는 실행 준비와 결과 확인이 포함되므로 실행 파일 자체의 순수 CPU 시간과 같지 않습니다.

단계별 일부 실패 기록은 종료 시각이 없어 해당 평균에서 제외하고 실제 표본 수를 표시했습니다. 전체 성공·실패 154개와 정리 상태 검증은 별도로 완료했습니다. 활성화 시각은 초 단위, 정리 ACK는 밀리초 단위입니다.

6. 최적화 이력 — 현재 설정과 과거 조건을 분리

같은 행 안의 입력·측정 경계가 맞는 조건끼리 비교한다. 서로 다른 문서 수와 계측 방법을 섞어 누적 개선 배수를 만들지 않는다. 아래 기록은 단회 시험이며, 실패·중단 기록은 완료 성능에서 제외한다.

초기 98쪽 시험은 현재 속도를 뜻하지 않는다

암호문 9,855,798바이트(9.40MiB), 98쪽·794청크. 초기 VM 8 vCPU·상한 50%, 임베딩 2개분 한도·2스레드에서는 약 843.53초가 걸렸다. 당시 GPU 14.70초와 약 57배 차이는 제한된 초기 배포 구성의 전체 시간 차이다. 현재 24개 OpenVINO/GPU 비교의 1.57배와 구분한다.

초기 임베딩 763초와 이후 129초는 VM 허용량을 두 배로 바꾼 것보다 큰 차이다. 한도·스케줄링·대기와 실제 실행이 함께 작용할 수 있지만, 이전 로그만으로 각 원인의 기여도를 확정하지 못했다. 763초 전체를 CPU 계산 시간 또는 Hyper-V 한 가지 원인의 결과라고 단정하지 않는다. 중간에 1→2개분으로 한도를 바꾼 첫 실패는 일정 조건의 실측으로 집계하지 않는다.

파서 1개를 유지했던 근거와 한계

같은 6문서·임베딩 4작업자	파서 1개	파서 2개
전체 색인 시간	81.75초	83.96초
파서 합산 CPU 시간 한도	논리 프로세서 4개분	논리 프로세서 4개분
임베딩 전체 작업자 동시 대기	10.33초	9.08초

당시에는 파서를 늘려도 대기가 1.25초만 줄어 파서 1개를 유지했다. 단회 측정이므로 파서 2개가 항상 느리다는 결론이 아니다. 문서 12슬롯·임베딩 12작업자로 확대했을 때 파서 1개가 최적인지도 이 시험으로 검증되지 않았다. 현재는 이후 별도 24개 시험을 거친 파서 2개를 사용한다. 원기록: parser2-sdpa/comparison.json, 같은 폴더 write-record.py.

같은 24개에서 작업자·CPU 영역·파서를 비교

24개 중 22개 성공·2개 OCR 제외, 242쪽·1,359청크. CPU 평균/최대는 호스트 전체 표본이며 VM 최대 허용량과 다르다.

과거 시험 구성	전체 초	호스트 CPU 평균/최대	그룹웨어 HTTP 평균/p95 ms
임베딩 4작업자·16개분 한도	92.13	16.33% / 24%	16.22 / 29.44
12작업자·48개분 한도	71.22	28.55% / 48%	16.68 / 30.29
24작업자·96개분 공유	77.05	28.42% / 74%	19.80 / 65.64
22작업자·88개분, 지원 영역 분리	76.45	28.08% / 80%	17.03 / 73.39
위 조건 + 파서 2개/합산 4개분	66.684	27.00% / 81%	18.09 / 31.71
파서 4개/합산 4개분	71.86	34.55% / 78%	14.78 / 19.87
파서 4개/합산 8개분	68.48	27.91% / 72%	20.17 / 88.33

48개분은 논리 프로세서 96개 중 48개에 해당하는 CPU 시간 한도다. 물리 CPU 칩 48개가 아니며 실제 사용률이 50%였다는 뜻도 아니다. 12작업자 시험의 실제 호스트 평균은 28.55%, 관측 최대는 48%였다.

허용량을 96개분으로 늘려도 입력 공급·문서별 길이·대기·긴 꼬리 때문에 빨라지지 않았다. 지원 영역 분리 시험은 작업자 수도 24→22로 바뀌므로 0.60초 차이를 CPU 영역 분리만의 효과로 주장하지 않는다. 66.684초는 당시 채택한 PyTorch 구성의 기록이며 현재 엔진은 OpenVINO다.

파싱이 빨라져도 전체가 같은 만큼 줄지 않는 이유

파서 4개에서 CPU 한도를 4→8개분으로 바꾸면 98쪽 문서의 파싱은 15.77→11.58초로 줄었지만 전체는 71.86→68.48초였다. 전체 파싱 종료가 63.12→60.18초, 이후 남은 구간이 8.75→8.30초였다. 그 문서의 파싱 후 대기는 9.18→10.67초, 임베딩은 15.63→15.56초였다. 단계가 겹치고 다른 문서의 종료를 기다리므로 파싱 절감량이 전체 시간에 그대로 더해지지 않는다.

마지막 임베딩 구간에는 22작업자 중 평균 약 4.24개 요청만 남았다. CPU 한도를 더 늘려도 마지막 일부 긴 요청을 자동으로 병렬 분할하는 것은 아니다. 이 수치는 해당 과거 시험의 꼬리 구간이며 최신 154개 전체의 평균과 구분한다.

배치를 바꾼 실험을 모두 운영에 적용하지 않았다

같은 24개 상세 계측	기본	요청을 최대 4개씩 분할	긴 청크만 별도 요청
전체 시간	70.159초	71.51초	65.11초
작업자 가동률	39.6%	43.9%	35.0%
호스트 CPU 평균/최대	28.64%/78%	29.73%/75%	26.20%/57%
HTTP 평균/p95 ms	19.93/35.99	28.27/119.16	15.39/20.85

긴 청크 분리 시험에서는 첫 임베딩 시작이 18.66→13.94초로 앞당겨졌지만 임베딩 전체 구간은 51.05→50.85초로 거의 같았다. 따라서 5초 단축을 분할 효과만으로 단정하지 않았다. 준비된 입력만 재생한 별도 실험에서는 30.03→29.27초, 가동률 75.5→91.3%였으나 CPU 사용 시간은 16.7% 늘었다. 이 29초는 복호화·파싱을 포함한 전체 색인 시간이 아니다.

채택 기준은 사용률을 최대화하는 것 자체가 아니라 전체 시간·검색 근거·메모리·그룹웨어 응답이다. 현재는 짧은 입력 최대 16개, 긴 입력 개별 계산, OpenVINO FP32를 사용한다. 긴 문서를 앞에 재정렬하지 않는다.

과거 GPU 대량 시험은 별도 기준

2026-10-15 GPU PC 시험은 244개·약 280.8MB, 223성공·OCR 제외 18개·기존 잘못된 파일 3개, 12,460청크였다. 워커 시작부터 최종 정리 ACK까지 283.436초였다. 호스트는 Ryzen 5 7600(6코어/12논리), GTX 1080 8GiB, RAM 약 15.16GiB. 입력과 시간 시작점이 현재 AVDC 시험과 달라 직접 배수를 계산하지 않는다.

이 기록의 문서·단계별 실행 시간을 합한 531.7초는 병렬로 겹친 시간을 중복 포함한다. 전체 283.4초와 모순이 아니다. GPU 평균 사용률은 56.0%였으며, 원본 해시 244개 유지·새 오류 없음·임시 평문 없음이 확인됐다. 이후 보고서의 24개 비교가 동일 입력 CPU/GPU 판단의 기준이다.

7. OpenVINO·청킹 품질과 확인하지 못한 원인

변경	확인한 결과와 한계
문서 병렬 처리	등록 순서를 유지하고 빈 슬롯에 다음 문서를 넣습니다. 앞 문서의 임베딩과 다음 문서 파싱을 겹쳐 전체 처리량을 높이는 방식입니다.
CPU 배분·작업자	스레드 수만 늘리지 않고 임베딩 요청을 여러 작업자로 분산했습니다. 입력 공급·긴 청크·복호화와 저장 대기 때문에 최대 한도까지 항상 쓰지는 않습니다.
OpenVINO FP32	같은 모델·입력·자원 한도로 엔진을 바꿨습니다. 24개 전체 시간 70.159 → 61.697초, 컨테이너 최고 메모리 41.92 → 31.78 GiB. INT8 양자화 효과가 아닙니다.
청킹 v3	Kordoc 추출 뒤에서 짧은 제목과 다음 본문을 묶습니다. Kordoc 자체를 수정한 청킹 변경은 아닙니다. 오늘 154개 청크 수는 2,135 → 1,596개였습니다.

OpenVINO가 빨라질 수 있는 이유

OpenVINO는 학습된 모델을 실행하는 엔진입니다. CPU 플러그인은 oneDNN 등의 계산 구현과 CPU의 병렬 실행 기능을 사용합니다. 모델과 하드웨어에 맞춘 실행·메모리 관리가 추론 비용을 줄일 수 있습니다. 기존 PyTorch에도 최적화가 있으므로 특정 기능 하나만으로 이번 차이를 설명하지 않습니다.

연산별 개선량·메모리 이동량·CPU 명령별 기여도는 분리 측정하지 않았습니다. 공식 문서의 일반 원리와 이 보고서의 실측 결과는 구분해야 합니다. [S4] OpenVINO CPU Device 문서.

파서 수와 검색 정확도는 제한된 표본의 검증입니다

과거 6문서·임베딩 4작업자에서 파서 1개 81.75초, 2개 83.96초였고 양쪽 파서 합산 한도는 4개분이었습니다. 전체 임베딩 작업자 동시 대기는 10.33 → 9.08초였습니다. 단회 결과로 파서 2개가 항상 느리다거나 현재 구성이 절대 최적이라고 결론 내리지 않습니다.

OpenVINO 전환 후 기존 12질문에서 문서·근거 순서가 일치했습니다. 청킹 v3의 별도 사례집 8질문에서도 첫 근거 적중 8/8을 확인했습니다. 오늘 154개 시험은 최종 검색 재평가를 병행하지 않았으므로 모든 문서의 검색 정확도 보장이 아닙니다.

같은 24개 비교의 단계별 상세값

단계 / 문서별 초	PyTorch 평균	OpenVINO 평균	OpenVINO 최대
원본 검증 / 24건	0.393	0.320	0.504
복호화 차례 대기 / 24건	2.480	2.693	9.912
복호화 호출·결과 검증 / 24건	0.835	0.634	1.190
파서 대기 / 24건	2.234	1.694	7.043
파서 내부 실행 / 24건	1.777	1.650	9.397
청크 생성 / 22건	0.241	0.271	1.219
임베딩·대기 / 22건	10.111	8.639	13.023
색인 저장 / 22건	0.643	0.390	2.824
활성화 / 22건	0.413	0.384	2.236
VM 임시 정리 / 24건	0.176	0.145	0.739
Windows 임시 정리 / 24건	0.055	0.057	0.212

복호화 실행 파일만의 OpenVINO 평균은 0.241초다. 호출·검증 0.634초와 구분한다. 단계 평균은 병렬 처리로 겹치며 합산하지 않는다. 154개 시험의 파서 API 왕복과 위 파서 내부 실행도 측정 경계가 다르다.

검색 근거가 달랐던 한 건

OpenVINO/PyTorch 1,359벡터의 최대 절대 차이는 1.63624e-6이었다. 별도 색인에서 달랐던 한 근거는 BM25의 샤드별 통계와 동점 정렬 차이로 추적했다. 전역 통계로 진단할 때 동점과 후보 일치가 확인됐으며, 이를 운영 검색 알고리즘 변경으로 적용한 것은 아니다. 이후 동일 source의 12질문에서 문서·근거·관련성 상태가 12/12 일치했다.

청킹 v3는 Kordoc 출력 뒤에서 제목과 본문을 묶는다. 사례집은 794→440청크, 48토큰 미만 44개·표 청크 108개, 8질문 첫 근거 적중 8/8 유지가 확인됐다. 이 표본이 모든 문서의 정확도를 보장하지는 않는다.

8. 엔진 전달 범위·코드·배포 위치

무엇을 만들었는가

등록된 Windows 폴더의 암호화 문서를 기존 uEncryptor2로 임시 복호화하고, Ubuntu VM의 Docker 서비스에서 파싱·청킹·임베딩·색인을 수행한다. 원본은 그대로 두고 작업용 평문은 정리한다. AVDC에서는 GPU와 OCR을 사용하지 않는다.

처리 흐름은 3장의 구조도를 따른다.

MySQL은 문서·버전·작업 상태를 관리한다. 문서별 진행이 겹치므로 파싱 중인 문서와 임베딩 중인 문서가 동시에 존재한다. 12슬롯은 동시에 처리할 문서의 상한이며 uEncryptor2를 12개 동시에 실행한다는 뜻은 아니다. 복호화는 공유 게이트로 제어한다.

검색은 저장된 청크와 벡터를 사용한다. KURE는 검색 후보의 순서를 다시 평가하는 외부 서비스이며, 문서를 색인하기 위한 임베딩 서비스와 역할이 다르다. 답변 생성 모델도 별도의 연결이다.

전달 범위와 책임 경계

구분	전달하거나 확인할 내용
엔진 인계 범위	소스와 버전, AVDC 설정 기준, 워커/API 연결 규격, 청킹·임베딩 동작, 시험 결과, 오류·정리 확인 방법
연동 담당자와 맞출 부분	프로젝트·source·폴더 매핑, 원본 경로, 실행 계정, 키 공급, 애플리케이션 인증, 실제 검색·답변 연결
운영 담당자 범위	VPN/방화벽/NAT/TLS, OS·VM 관리, 부팅 후 자동 실행, 백업·복구·모니터링·용량 관리
별도 자산	Docker 이미지·모델 가중치·uEncryptor2 및 DLL·실제 설정·인증키. Git만으로 모두 복원할 수 없음

운영 항목은 엔진 미구현 기능과 구분한다. 다만 아래의 source 초기 등록이나 신규 환경 재현처럼 엔진을 연결하는 데 필요한 미완료 항목은 숨기지 않고 인계한다.

코드와 파일 위치

저장소 경로	역할
tools/windows/Start-DocMindIngestionPool.ps1	최대 12슬롯 Windows 색인 워커 실행
tools/windows/Start-DocMindUEncryptorHostWorker.ps1	작업 claim, 검증, 복호화, 전달, 임시 파일 정리
tools/windows/DocMindUEncryptorHostWorker.Common.ps1	HMAC·경로·정리 공통 처리. 워커와 같은 버전으로 전달
tools/windows/DocMindDecryptGate.ps1, DocMindIngestionPool.ps1	공유 복호화 제어, 자식 프로세스 관리
tools/windows/Watch-DocMindEncryptedSources.ps1	원본 조사·변경 감지. 현재 AVDC 상시 감시 검증과는 구분
app/api/apps/restful_apis/docmind_api.py	워커·문서·검색 API
app/api/apps/services/docmind_ingestion_service.py	문서 버전·작업 임대·색인 활성화·정리 상태
app/rag/parser_platform/office_chunker.py, config.py	Kordoc 이후 청킹 및 버전 fingerprint
tools/avdc_embedding/	AVDC OpenVINO 서비스·풀·변환 코드·설정 예제
tools/local_embedding/	별도의 기존 PC GPU 경로. AVDC 코드로 덮어쓰지 않음

AVDC 배포 폴더는 아래와 같다. 11월 10일 인계서 보완 중 고정 호스트키 SSH로 8개 서비스 running, 이미지 ID·자원 설정과 모델·핵심 소스 해시를 다시 읽기 전용 확인했다. Windows 복호화 모듈은 이번에 재해시하지 않았으므로 인수 때 확인한다.

위치	내용
Windows D:₩DocMind₩worker₩scripts	워커 코드 묶음
Windows D:₩DocMind₩worker₩runtime	서버별 워커 설정·실행 파일·정리 영수증
Windows D:₩DocMind₩worker₩decryptor	uEncryptor2 및 의존 DLL
Windows D:₩DocMind₩tmp	임시 평문 공간. 원본 저장소가 아님
Ubuntu /opt/docmind-avdc/runtime	실제 API·임베딩 소스
Ubuntu /opt/docmind-avdc/config/compose.json	병합된 실행 구성. 비밀값이 있어 Git에 올리지 않음
Ubuntu /opt/docmind-avdc/config/embedding.json	임베딩 작업자·스레드 설정
Ubuntu /opt/docmind-avdc/models/openvino	변환 모델 model.xml, model.bin
Ubuntu /opt/docmind-avdc/manage.sh	고정 프로젝트·Compose를 사용하는 관리 명령

기존 D:₩uPLEXsoft₩uDrive₩DEPT 원본은 별도다. Git에는 원본·청크·벡터·실제 .env·키·DB 백업을 넣지 않는다. 예전 설치 안내와 이 문서가 다르면 AVDC의 현재 설정은 이 문서 및 연결한 최신 배포 기록을 기준으로 한다. 10월 Windows 실행 ZIP이 최신 AVDC 설정을 포함한다고 가정하면 안 된다.

9. 수령할 실행 자산과 전달·복원 방법

2026-11-10 AVDC에서 읽기 전용으로 조회했다. 정확한 이미지 ID, 자원 한도, 모델·핵심 소스 SHA256은 assets.json에 있다. 이 목록은 실제 확인한 자산의 목록이며 **다운로드 가능한 새 설치 ZIP을 만들었다는 뜻이 아니다**.

1. Git으로 받는 것

GitLab rlaxortj00/docmind의 main을 받고 실제 받은 git rev-parse HEAD를 인수 기록에 적는다. GitLab 접근 토큰은 승인된 credential helper로 제공하며 URL에 넣지 않는다.

명령/데이터 · bash

```
git clone --branch main http://128.134.187.123:9010/rlaxortj00/docmind.git
cd docmind
git rev-parse HEAD
```

- API·청커·파서 연결·Windows 워커·OpenVINO 풀 소스.
- 설정 예제, 연동 명세, 실행/합격 절차, 합성 문서 생성·상태 확인 도구.
- 모델 변환 코드 및 버전 고정 기록. 모델 바이너리·컨테이너 이미지 자체는 별도다.

2. 별도로 전달할 것

자산	현재 위치/식별자	수령 확인
API 이미지	AVDC Docker docmind-avdc:cpu-noocr-20261102	이미지 ID 6efabc34..., 전체 값 assets.json
OpenVINO 이미지	docmind-gte-openvino:20261106	이미지 ID 5b54a0e5...
파서 이미지	docmind-kordoc-parser:2026.10.19	이미지 ID 53fc3d95...
ES·MySQL·Valkey·MinIO·preview 이미지	각각 assets.json의 configured_image	tag만이 아니라 ID 대조
API overlay 소스	/opt/docmind-avdc/runtime/api	**기반 API 이미지와 함께 필요**. 이미지 이름 날짜만으로 최신 청킹 포함 여부 판단 금지
임베딩 소스	/opt/docmind-avdc/runtime/embedding	service/pool SHA256 대조
OpenVINO 모델	/opt/docmind-avdc/models/openvino/model.xml 및 model.bin	두 파일 모두 assets.json SHA256 일치, bin 약 1.26GB
토크나이저·원모델·신뢰 코드 캐시	임베딩 기반 이미지의 /opt/docmind/model 및 모델 캐시	모델 revision 9bbca17d9273fd0d03d5725c7a4b0f6b45142062, 코드 revision 40ced75c3017eb27626c9d4ea981bde21a2662f4
실제 Compose·설정	/opt/docmind-avdc/config, manage.sh	환경별 경로·마운트·이미지·포트 대조. 실제 설정은 별도 비공개 전달
키	Ubuntu secrets 및 Windows 워커 설정이 가리키는 키	HMAC·임베딩·세션·DB 키를 용도별 전달, Git 업로드 금지
Windows 복호화 모듈	AVDC D:₩DocMind₩worker₩decryptor	exe와 의존 DLL을 함께, 공급사 배포 조건 준수
워커 실제 설정·코드	D:₩DocMind₩worker₩runtime, scripts	원본 루트·API loopback·공유 복호화 게이트·실행 계정 확인

기존 uEncryptor2의 승인 기록은 복호화 검증의 버전 1.0.0.2, SHA256 8f1506520127011b0c425121fa583bcc9c1533b60ca9cd28540d45b5c645d457이다. **이번에는 Windows 바이너리를 재해시하지 않았다.** 인수 시 실제 exe의 해시·Authenticode를 확인한다. DLL 해시와 배포 권한도 별도 인수 목록에 적는다.

현재 모델/이미지를 그대로 전달하는 경로와 새로 빌드하는 경로를 구분한다. OpenVINO Dockerfile/변환 안내는 기존 기반 이미지와 오프라인 wheels를 전제로 한다. 빈 서버에서 전체 빌드를 검증한 새 릴리스가 아니다.

3. 이미지·파일 전달을 준비할 때

아래는 자산 담당자가 실행할 **선택적 내보내기 절차**다. 이번 문서 작업에서 대용량 export나 서비스 재기동은 하지 않았다. 새 디렉터리를 쓰고 디스크 여유를 먼저 확인한다. Compose의 실제 이미지 이름이 assets.json과 다른 경우 먼저 원인을 확인한다.

명령/데이터 · bash

```
# Ubuntu AVDC, 기존 관리 폴더 권한을 가진 계정
sudo /opt/docmind-avdc/manage.sh images
df -h /opt/docmind-avdc
```

승인된 전달 디렉터리에서 Docker save로 다음 **8개 이미지**를 하나의 archive로 내보낸다. 이 명령은 실행 중인 컨테이너를 중지하지 않지만 디스크/IO를 사용한다.

명령/데이터 · bash

```
# /secure-transfer는 담당자가 준비한 전용 경로로 바꾼다.
# 기존 결과를 덮어쓰지 않도록 존재 여부를 확인한다.
test ! -e /secure-transfer/docmind-images.tar || exit 1
sudo docker save -o /secure-transfer/docmind-images.tar ₩
  docmind-avdc:cpu-noocr-20261102 docmind-gte-openvino:20261106 ₩
  docmind-kordoc-parser:2026.10.19 docmind-preview:2026.10.19 ₩
  elasticsearch:8.11.3 mysql:8.0.40 valkey/valkey:8 ₩
  pgsty/silo:RELEASE.2026-08-06T00-00-00Z
sha256sum /secure-transfer/docmind-images.tar
```

출력 tar의 SHA256은 **새로 생성한 파일의 전송 확인용**이며 assets.json의 Docker image ID와 다른 값이다. runtime 코드·OpenVINO 모델·설정·키는 별도 전달한다. 특히 /opt/docmind-avdc 전체를 무작정 압축하면 운영 DB·원문 파생물·키까지 포함될 수 있다. 필요한 디렉터리만 선별한다.

수신 서버에서 archive SHA256을 송신 측과 대조한 뒤 docker load -i <전달받은 tar>를 실행하고 docker image inspect <tag> --format '{{.Id}}'를 assets.json과 비교한다. 모델은 sha256sum model.xml model.bin, Windows 파일은 Get-FileHash -Algorithm SHA256으로 확인한다.

4. 새 서버와 기존 AVDC의 차이

- **기존 AVDC 인수:** 현재 이미지·마운트·프로젝트가 준비돼 있다. RUNBOOK의 읽기 확인 및 새 시험 source 등록부터 진행한다.

- **새 서버 설치:** 위 자산과 서버별 설정·빈 인프라·마운트 경로를 먼저 준비한다. 현재 AVDC Compose는 민감 설정을 포함하므로 통째로 공개하거나 다른 서버에서 바로 실행하지 않는다.

- 기존 Windows 패키지의 deploy/windows/scripts/bootstrap.py는 모델·공유 작업공간·source 초기화를 수행한다. Initialize-DocMind.ps1/Start-DocMind.ps1과 묶인 **해당 패키지의 새 설치**에 사용한다. 이미 운영 중인 AVDC에 임의 실행하면 tenant 모델 기본값까지 바뀔 수 있다.

- 이번 인계서 보완은 새 OpenVINO 설치 ZIP 제작·Windows 상용 모듈 전달·새 서버 전체 설치 시험까지 수행한 작업이 아니다. 수령자가 실행하기 위한 전제 자산과 다음 명령을 구체화한 것이다.

10. 실행 설정 작성과 확인

실제 값은 Git 밖에 둔다. 아래는 키 값이 아닌 **이름·형식·확인 방법**이다. 기존 AVDC에서는 현재 제한된 설정 파일을 인계받아 사용하며, 새 키를 임의로 발급해 기존 값을 덮어쓰지 않는다.

Windows 워커

JSON 예제를 복사한다. 필요한 전체 워커 소스는 tools/windows 묶음이다. 단일 ps1만 복사하면 Common·게이트·C# 감시기 의존 파일이 빠질 수 있다.

필드	입력 / 확인
worker_id	영문·숫자·_.:- ID, 최대 128자. 12슬롯이면 코드가 슬롯 접미사를 붙임
api_base_uri	Windows에서 도달 가능한 loopback HTTP(S) API. 예: http://127.0.0.1:19380. 실제 전달 포트와 대조
key_id	API의 DOCMIND_HOST_WORKER_KEY_ID와 같아야 함
shared_secret_file	32바이트 HMAC 키 파일. API에 마운트한 동일 키와 대응
executable_path	해당 서버 uEncryptor2.exe. 실제 서명·해시 확인
executable_sha256	승인된 실행 파일의 SHA256. 새 바이너리면 재검증 후 지정
executable_signer_thumbprint	승인된 서명 인증서 thumbprint. 해시와 함께 대조
work_root	원본과 분리한 작업용 평문 폴더. 시험은 전용 빈 폴더 사용
cleanup_receipt_root	정리 ACK 영수증. 평문 폴더와 분리, 재시도 전에 삭제하지 않음
decrypt_control_root	기존 워커와 같은 공유 복호화 게이트 경로. 달리 잡아 동시 실행 제한을 우회하지 않음
sync_state_root	감시기 SQLite 상태, source 원본 밖. 운영 상태를 시험용으로 복사/삭제하지 않음
sources	[{"source_id":"handoff-sample","root":"D:₩₩ApprovedEncryptedSampleRoot"}]. 서버 논리 source 등록과 동일 ID
job_timeout_seconds	복호화 CLI 제한, 예제 120초. 전체 색인 제한이 아님
claim_lease_seconds	전체 작업 임대, 예제 1,800초. 서버 범위 30~1,800초
lease_safety_seconds	전달·정리 안전 여유, 예제 10초
cleanup_receipt_replay_seconds	ACK 재전송 주기, 예제 60초
reaper_minimum_age_seconds	만료 작업 회수 최소 나이, 예제 300초
preview_enabled	색인 전용 인수 시험은 false. preview는 별도 역할
require_bitlocker	예제 true. 실제 보호 볼륨 상태와 일치해야 함. 실패를 숨기려고 낮추지 않음
backup_exclusion_acknowledged	실제 임시 폴더 백업 제외 여부를 확인한 후 true. 폴더 생성만으로 제외되는 것은 아님
scan_batch_size	예제 250. 전체 정합성 스캔의 배치 크기

필드	입력 / 확인
daily_reconciliation_timezone/local_time	예제 Asia/Seoul / 00:00. 상시 운영 작업 등록은 별도

키 파일 ACL은 실행 계정·SYSTEM 등 필요한 주체로 제한한다. 기존 키 생성 도구는 저장소 .local 개발 환경용이다. AVDC 기존 키를 회전하는 용도로 그대로 실행하지 않는다.

API·파서·임베딩

위치 / 이름	기준값·의미
API DOCMIND_HOST_WORKER_KEY_ID	Windows key_id와 일치
API DOCMIND_HOST_WORKER_HMAC_SECRET_FILE	컨테이너 내부 키 파일 경로. Windows와 같은 32바이트 키
API DB/ES/Valkey/스토리지 설정	현재 Compose·서비스 설정과 일치. 새 설치는 새 인증값, 기존 볼륨에는 기존 값 유지
API DOCMIND_SHARED_WORKSPACE_ENABLED	현재 AVDC 1. 공용 작업공간 설치이며 사용자별 ACL 검증 완료로 확대 해석하지 않음
API 공유 사용자	기존 설치의 공유 사용자 설정과 DB 계정이 대응해야 bootstrap 가능
API 문서 동시성	DOCMIND_INGESTION_DOCUMENT_CONCURRENCY=12
API 임베딩 요청 상한	MAX_CONCURRENT_EMBEDDING_REQUESTS=24
API 파서 동시성	KORDOC_CLIENT_CONCURRENCY=2
파서 프로세스 수	KORDOC_WORKERS=2, 합산 CPU 한도 4개분, 메모리 1GiB
API 청킹 버전	PARSER_PLATFORM_KORDOC_CHUNKER_REVISION=docmind-kordoc-chunker-v3
임베딩 설정	device=cpu, threads=4, cpu_short_batch=16, cpu_groups 22개
임베딩 인증	api_key_file로 별도 Bearer 키 공급. HMAC 키와 혼용하지 않음
OpenVINO 모델	model.xml/bin 모두 필요, FP32·768차원, tokenizer/신뢰 코드 캐시는 기반 이미지에 있음
외부 검색	KURE 키·통신 경로는 별도. 색인 시험에 KURE 호출은 필요하지 않음
답변 생성	답변 모델 URL·인증은 별도. 이번 합성 색인 합격 조건에 포함하지 않음

96 vCPU보다 작은 환경에서는 AVDC의 CPU 번호를 그대로 복사하지 않는다. 컨테이너 메모리 합계·VM 메모리·호스트 여유를 함께 확인한다. Compose에 cpus: 88이 있어도 항상 서버 전체 CPU의 88%를 사용하는 뜻은 아니다.

설정 인계 시 채울 비공개 표

키 자체 대신 다음 위치만 승인된 채널로 전달한다: 실제 Compose 경로, 워커 JSON 경로, HMAC 키 공급 위치, 임베딩 키 공급 위치, source/project ID, Windows 원본 루트, API 로컬 포트, 실행 계정, 임시 작업·영수증·동기화 상태 경로. 본 저장소의 예시 경로는 자동 생성된 실제 경로가 아니다.

11. Windows 워커와 엔진 API 연동

기준은 app/api/apps/restful_apis/docmind_api.py, docmind_ingestion_service.py, docmind_worker_auth.py다. 아래 ID·해시·시간은 예시이며 그대로 송신하는 테스트 데이터가 아니다. 암호화 원본 연결은 기존 Windows 워커를 사용하는 것이 기본이다.

1. 인증과 응답 검증

접두사는 /api/v1. HMAC 키는 32바이트이며, API는 원시 바이트 또는 64자리 hex/base64 파일을 읽는다. 같은 키를 Windows 워커에 공급한다. 타임스탬프는 Unix 초, 허용 시계 차이는 300초다.

명령/데이터 · text

```
X-DocMind-Key-Id: <등록된-key-id>
X-DocMind-Timestamp: <Unix 초>
X-DocMind-Nonce: <요청마다 새로운 nonce>
X-DocMind-Content-SHA256: <실제로 전송한 본문 바이트의 소문자 SHA256>
X-DocMind-Signature: <HMAC-SHA256 소문자 hex>
```

서명 원문은 아래 다섯 필드를 LF로 연결한다. 마지막에 줄바꿈을 덧붙이지 않는다. 메서드는 대문자, 경로에는 /api/v1과 실제 쿼리를 포함한다. JSON을 서명한 뒤 다시 직렬화하면 해시가 달라질 수 있다.

명령/데이터 · text

```
METHOD₩nPATH_AND_QUERY₩nTIMESTAMP₩nNONCE₩nCONTENT_SHA256
```

응답에도 같은 이름의 서명 헤더가 온다. 받은 원문 바이트의 해시·서명·시간·응답 nonce를 검증한 뒤 JSON을 사용한다. 검증 없이 accepted: true만 믿지 않는다. 인증 실패 401 응답은 서명되지 않을 수 있으므로 실패로만 처리한다. 정상 ACK의 job/version/fence도 원래 임대와 대조한다. 자세한 응답 canonical 처리는 기존 공통 라이브러리를 기준으로 한다.

2. 원본 버전 관찰

POST /api/v1/cloud-sync/host-worker/observations

명령/데이터 · json

```
{"source_id":"handoff-sample","document_id":"11111111111111111111111111111111","relative_path":"sample.docx","ciphertext_sha256":"aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa","size":4096,"mtime_ns":1794272400000000000}
```

정확히 위 필드가 필요하고 size, mtime_ns는 정수다. 이 API는 이미 등록된 문서의 버전 관찰용이다. 처음 발견한 파일의 문서 생성은 감시기의 세션·변경·스캔 프로토콜을 사용한다. 새 개발자가 observations로 신규 문서를 임의 생성하면 안 된다.

안정 관찰 전 응답 예: {"state":"WAITING_SOURCE_STABLE","stable_observations":1}. 등록 가능한 상태에서는 {"state":"DISCOVERED","version_id":"…","job_id":"…"}. 이전 작업 정리가 남으면 DEFERRED_PREVIOUS_CLEANUP일 수 있다. 전송 성공과 작업 생성은 구분한다.

3. 작업 임대와 유효성 확인

POST /api/v1/cloud-sync/host-worker/claim

명령/데이터 · json

```
{"worker_id":"handoff-worker","protocol_version":1,"lease_seconds":1800,"claim_source_id":"handoff-sample"}
```

응답 예:

명령/데이터 · json

{"job":{"job_id":"22222222222222222222222222222222","source_id":"handoff-sample","document_id":"11111111111111111111111111111111","version_id":"33333333333333333333333333333333","relative_path":"sample.docx","ciphertext_sha256":"aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa","fencing_token":1,"lease_expires_at":"2026-11-10T03:30:00Z"}}

작업이 없으면 {"job":null}이며 실패가 아니다. lease_seconds는 30~1,800초. 풀의 ingestion_pool_size는 1~12이며 실제 서버 슬롯 상한과 함께 작동한다. claim_job_id는 기존 특정 시험용 제약이 있으므로 범용 작업 지정 API로 사용하지 않는다.

POST /api/v1/cloud-sync/host-worker/jobs/{job_id}/validate

명령/데이터 · json

{"worker_id":"handoff-worker","version_id":"33333333333333333333333333333333","fencing_token":1}

정상: {"valid":true,"lease_expires_at":"2026-11-10T03:30:00Z"}. 이 요청은 임대를 갱신하는 기능이 아니다.

4. 평문 전달

PUT /api/v1/cloud-sync/host-worker/jobs/{job_id}/artifact

본문은 JSON이나 multipart가 아니라 **복호화한 파일의 원시 바이트**다. 기본 HMAC 헤더에 아래를 추가한다.

명령/데이터 · text

```
Content-Type: application/octet-stream
Content-Length: <정확한 바이트 수>
X-DocMind-Worker-Id: <claim한 worker_id>
X-DocMind-Version-Id: <version_id>
X-DocMind-Fencing-Token: <정수 fence>
X-DocMind-Plaintext-SHA256: <평문 SHA256>
X-DocMind-Plaintext-Size: <정확한 바이트 수>
```

Content-Length는 필수(없으면 411). API 기본 최대 256MiB이며 파서의 별도 입력 크기·형식 제한도 적용된다. 256MiB가 모든 문서의 처리 보장 크기는 아니다. 요청은 파싱·색인·서버 정리까지 기다릴 수 있다. CLI 복호화 timeout과 별개로 남은 lease 안에서 기다린다.

성공 ACK: {"accepted":true,"job_id":"...","version_id":"...","fencing_token":1}. 이때에도 호스트 파일을 지우고 정리 상태를 보고해야 전체 완료다. 네트워크가 끊겼다고 같은 artifact를 무조건 재전송하지 않는다. 이미 활성화됐거나 임대가 바뀌었을 수 있다.

5. 호스트 정리·ACK

POST /api/v1/cloud-sync/host-worker/jobs/{job_id}/status

명령/데이터 · json

{"worker_id":"handoff-worker","version_id":"33333333333333333333333333333333","fencing_token":1,"status":"CLEANED","error_code":null}

정상 ACK: {"accepted":true,"job_id":"...","version_id":"...","fencing_token":1,"cleanup_required":false}. CLEANED는 호스트 파일이 실제로 사라진 뒤 보낸다. 기존 워커는 별도 정리 receipt를 먼저 남기고 서명된 ACK 후 영수증을 지워 통신 실패 때 재전송한다. FAILED는 실패했지만 호스트 정리는 끝난 경우, CLEANUP_FAILED는 정리 실패다. ACTION_REQUIRED는 사용자 조치가 필요한 상태다.

POST .../jobs/{job_id}/cleanup/prepare는 **만료된 임대이며 서버 정리는 끝났고 호스트 정리만 남은 작업의 복구용**이다. validate와 같은 worker/version/fence 본문을 사용한다. 평상시 성공 흐름에서 필수 호출하는 API가 아니다. 임의 파일 삭제 권한을 얻는 용도로 사용하지 않는다.

6. 상태 전이·실패 처리

상태: DISCOVERED / RETRY_WAIT → DECRYPTING → PARSING → INDEXING → CLEANUP → COMPLETE. 실패는 FAILED / ACTION_REQUIRED / CLEANUP_FAILED로 구분한다.

원본이 바뀌거나 삭제되면 SUPERSEDED / DELETED도 가능하다. 성공 여부는 lifecycle 하나뿐 아니라 서버·호스트 cleanup, 활성 버전·청크 세트·ES 개수까지 대조한다.

HTTP/상태	처리 원칙
401 / WORKER_AUTH_FAILED	키·시계·직렬화·서명 확인. 키가 틀린 상태로 반복하지 않음
409 / stale lease·result·cleanup	소유 worker/version/fence와 원본 버전 대조. 새 임대는 기존 claim 경로로만 획득
411 / 413	본문 길이·상한 수정. 그대로 재시도하지 않음
503 / DOCMIND_INGESTION_BUSY, 임베딩 queue busy	기존 제한된 재시도/큐 경로 사용. 현재 시도의 결과·정리부터 확인
OCR 필요 / 형식·토큰 한도 오류	입력 조건 문제로 분류. 이 비OCR 구성에서 성공으로 처리하지 않음
timeout / 500 / ACK 유실	작업 상태·원본 세대·receipt 확인. 서버 상태를 직접 COMPLETE로 바꾸지 않음
CLEANUP_FAILED	정리를 먼저 복구하고 성공 재처리는 그 다음. 기록/임대/영수증을 삭제해 우회하지 않음

7. 조회·검색과 삭제 경계

워커 /status는 POST 보고용이며 GET 조회가 없다. 인수 시에는 CLI의 jobs, check-job으로 읽기 전용 확인한다. 애플리케이션 문서 상세·검색은 사용자 인증/공유 작업공간 인증과 tenant 범위를 사용한다. 워커 HMAC는 로그인 대신 쓰지 않는다.

엔진 색인에서 파일 삭제를 반영하는 것은 **원본 감시기의 확정 삭제 이벤트** 경로다. DELETE /docmind/documents/{id}는 원본 시스템 쓰기 연동을 수반할 수 있으므로 시험 정리용으로 사용하지 않는다. 합성 시험 후에는 감시기·워커를 정상 종료하고 별도 source 결과를 보관해 인수자가 확인한다. 전량 삭제나 운영 DB 직접 변경은 본 절차에 없다.

임베딩 연동

임베딩 풀 내부 주소에 GET /health를 보내면 status, model, dimensions, device, workers를 확인할 수 있다. 정상 기준은 HTTP 200, dimensions: 768, device: cpu, workers: 22다. 모든 작업자가 준비되기 전에는 503이 반환될 수 있다.

명령/데이터 · http

```
POST /v1/embeddings
Authorization: Bearer <별도 공급한 임베딩 키>
Content-Type: application/json

{"model":"gte-multilingual-base","input":["합성 문서 확인 문장"]}
```

응답은 입력 순서에 대응하는 data[].index, data[].embedding과 사용 토큰 정보를 제공한다. 벡터는 각각 768차원이며 길이별 계산 후에도 요청의 원래 순서를 복원한다. 키는 헤더로 공급하고 Git·명령 이력·공용 로그에 저장하지 않는다. 메

모리 부족이나 큐 포화 503을 색인 성공으로 처리하지 않는다.

문서·검색 연동

문서 상세는 GET /api/v1/docmind/documents/{document_id}, 검색은 POST /api/v1/docmind/search다. 사용자 로그인·tenant/project 범위를 사용하며 워커 HMAC 키로 로그인 인증을 대신할 수 없다.

명령/데이터 · json

```
{
  "question": "합성 문서의 확인 문장은 무엇인가요?",
  "project_id": "<프로젝트-ID>",
  "scope": {"mode": "documents", "document_ids": ["<문서-ID>"]}
}
```

HTTP 성공만 보지 말고 응답의 업무 결과와 근거 문서·청크를 확인한다. KURE 외부 연결과 별도의 답변 모델이 준비되지 않은 상태에서, 색인 완료를 검색·답변 전체 연결 완료로 보고하면 안 된다. 원문 미리보기 역시 별도의 preview 워커·권한·임시 접근 수명이 필요하다.

12. 개발자 실행 순서와 합격 확인

목표: **합성 DOCX 한 건으로 원본 복호화 → 파싱 → 청킹 → 임베딩 → 활성 색인 → 임시 파일 정리 확인**. 기존 문서·색인을 지우지 않는다. 이 절차는 담당자가 인수 때 실행하는 절차이며, 아래 기대 결과를 이번에 새 색인해 얻었다는 뜻은 아니다.

필수 전제: 실행 자산, 설정, 준비된 API·DB·파서·임베딩, Windows 실행 계정과 정상 uEncryptor2, 실제 암호화된 합성 표본. 새 서버에서 이 전제가 비어 있으면 먼저 자산 수령·초기화를 한다.

A. 현재 서비스와 도구 준비 — Ubuntu VM

명령/데이터 · bash

```
sudo /opt/docmind-avdc/manage.sh ps
```

기대: 8개 서비스가 running. healthcheck가 정의된 서비스는 healthy. 시작이 필요한 준비된 설치에서만 sudo /opt/docmind-avdc/manage.sh up -d를 실행한다.

Git checkout의 tools/handoff를 API 컨테이너의 임시 경로에 복사한다. 아래 DOCMIND_REPO는 실제 clone 경로다. **쉘은 Ubuntu Bash**다.

명령/데이터 · bash

```
DOCMIND_REPO=/path/to/docmind
API_CONTAINER=$(sudo /opt/docmind-avdc/manage.sh ps -q docmind)
test -n "$API_CONTAINER" || exit 1
sudo docker exec "$API_CONTAINER" test ! -e /tmp/docmind-handoff || exit 1
sudo docker cp "$DOCMIND_REPO/tools/handoff" "$API_CONTAINER":/tmp/docmind-handoff
```

한 세션에서 다음 함수로 도구를 호출한다. 새 터미널을 열면 변수와 함수를 다시 선언한다.

명령/데이터 · bash

```
dm_handoff() {
  sudo docker exec -w /ragflow -e PYTHONPATH=/ragflow "$API_CONTAINER" ₩
    /ragflow/.venv/bin/python /tmp/docmind-handoff/engine.py "$@"
}
dm_handoff projects
```

기대: project_id, dataset_id, mode: database, source 개수 목록. 비어 있으면 초기 작업공간이 없는 설치다. 임의 ID를 만들지 않는다. 신규 Windows 패키지는 해당 패키지 초기화 절차를 사용하고, 기존 AVDC에서는 기존 프로젝트 ID를 인계받는다.

B. 프로젝트·폴더·source 최초 등록

기존 공용 작업공간을 사용하는 API에서 workspace가 아직 없다면, 기존 공유 사용자와 모델 등록이 먼저 완료된 상태에서 아래 기존 서비스를 사용한다. 이 명령은 **필요한 신규 설치에서만** 실행하며 기존 모델·사용자를 새로 만들지 않는다.

명령/데이터 · bash

```
sudo docker exec -w /ragflow -e PYTHONPATH=/ragflow "$API_CONTAINER" /ragflow/.venv/bin/python -c ₩
  'from common import settings; settings.init_settings(); from api.apps.services.docmind_shared_workspace_service import resolve_user;
from api.apps.services.docmind_bootstrap_service import bootstrap_shared_workspace; import json;
print(json.dumps(bootstrap_shared_workspace(resolve_user().id)))'
```

기대: project_id, dataset_id, created, catalog_initialized. 빈 workspace의 catalog_initialized: false 자체는 실패가 아니다. 공유 사용자/tenant 임베딩 미설정 오류는 설치 초기화에서 해결한다. 이를 우회해 임의 SQL로 프로젝트를 만들지 않는다.

다음은 **기존 프로젝트에 새로운 시험 source와 기본 폴더만** 추가한다. 모델·tenant·기존 문서·색인은 바꾸지 않는다.

명령/데이터 · bash

```
PROJECT_ID='<projects에서 확인한 ID>'
SOURCE_ID=handoff-sample-20261110
dm_handoff register-source --project "$PROJECT_ID" --source "$SOURCE_ID" --display-name '엔진 인수 합성 시험'
# action: would_create와 project/source를 확인한 뒤에만 실행
dm_handoff register-source --project "$PROJECT_ID" --source "$SOURCE_ID" --display-name '엔진 인수 합성 시험' --apply
```

기대: 첫 실행 created, 같은 등록을 다시 요청하면 unchanged. 비활성/다른 프로젝트/이름 충돌은 오류로 멈추며 기존 값을 덮어쓰지 않는다. 여러 source는 서로 다른 기본 폴더 ordinal로 등록한다. 경로 추가는 다음 Windows 설정에서 수행한다.

C. 합성 입력 준비 — Windows

Python 3가 있는 checkout 루트에서 원본 업무 폴더 밖의 전용 작업 디렉터리를 만들고 실행한다. 출력 파일이 이미 있으면 덮어쓰지 않는다.

명령/데이터 · powershell

```
New-Item -ItemType Directory -Path 'D:₩DocMind₩management₩handoff-sample-20261110' -ErrorAction Stop
python .₩tools₩handoff₩make_sample.py --output 'D:₩DocMind₩management₩handoff-sample-20261110₩sample.docx'
```

출력의 plaintext_sha256을 인수 기록에 저장한다. 문서에는 DOCMIND-HANDOFF-20261110 표식과 비민감한 확인 문장만 들어 있다.

이 DOCX는 아직 평문이다. uDrive 등 기존 제품의 검증된 업로드·암호화 경로로 시험용 저장 폴더에 넣어 암호화 원본을 준비한다. /enc 등 확인되지 않은 CLI 옵션을 추측해서 실행하지 않는다. 이미 공급자가 준비한 합성 암호문/평문 한 쌍이 있으면 그 파일과 독립적으로 구한 평문 SHA256을 써도 된다. 암호화 준비가 안 됐으면 여기서 미준비로 기록하며 복호화 검증을 건너뛰고 전체 합격으로 처리하지 않는다.

원본 암호문의 SHA256을 기록한다.

명령/데이터 · powershell

```
$encryptedSample = '<암호화가 확인된 sample.docx 전체 경로>'
$cipherBefore = (Get-FileHash -LiteralPath $encryptedSample -Algorithm SHA256).Hash
```

D. 시험용 워커 설정 연결

설정표에 따라 기존 인증·복호화 설정을 사용하는 별도 handoff-worker.private.json을 Git 밖에 만든다. sources는 시험 source 하나와 **암호화된 합성 표본만 있는 루트**로 제한한다. 평문 DOCX를 원본 루트에 직접 넣지 않는다.

- source_id는 B의 SOURCE_ID와 같게 한다.

- work_root, cleanup_receipt_root, sync_state_root는 다른 작업과 분리한 시험 경로를 쓴다.

- decrypt_control_root는 같은 호스트의 기존 워커와 공유한다.

- 기존 색인 워커와 인수 워커를 중복 실행하지 않는다. 진행 중이면 기존 작업 정리·종료를 확인한 뒤 시험한다.

- 현재 계정의 실제 uEncryptor2 해시·서명, 임시 볼륨 보호·백업 제외 조건을 확인한다.

별도 Windows PowerShell 세션 두 개에서 실행한다. 모두 **checkout 루트 기준**이다.

명령/데이터 · powershell

```
# 세션 1: 최초 조사와 이후 안정화 관찰
$handoffConfig = 'D:₩DocMind₩management₩handoff-sample-20261110₩handoff-worker.private.json'
$watchStop = 'D:₩DocMind₩management₩handoff-sample-20261110₩watch.stop'
powershell.exe -NoProfile -File .₩tools₩windows₩Watch-DocMindEncryptedSources.ps1 -ConfigPath $handoffConfig -SourceId
handoff-sample-20261110 -StopFile $watchStop
```

명령/데이터 · powershell

```
# 세션 2: 인수 확인은 1슬롯. 성능 측정이 아님
$handoffConfig = 'D:₩DocMind₩management₩handoff-sample-20261110₩handoff-worker.private.json'
$ingestStop = 'D:₩DocMind₩management₩handoff-sample-20261110₩ingest.stop'
powershell.exe -NoProfile -File .₩tools₩windows₩Start-DocMindIngestionPool.ps1 -ConfigPath $handoffConfig -Concurrency 1
-ClaimSourceId handoff-sample-20261110 -StopFile $ingestStop
```

기존 stop 파일이 있으면 새 시도 경로를 정한다. 감시기는 안정화·독립 관찰 때문에 바로 작업을 만들지 않을 수 있다. 수정 이벤트는 기본 120초 안정화와 이후 독립 관찰을 거치므로 생성 직후 0건만 보고 실패로 결론 내리지 않는다. 초기 스캔·세션 복구 대기는 별도다. -Once를 붙여 전체 조사 완료를 가정하지 않는다.

E. 색인·정리 합격 확인 — Ubuntu와 Windows

Ubuntu A의 Bash 세션에서:

명령/데이터 · bash

```
dm_handoff jobs --source "$SOURCE_ID"
JOB_ID='<이번 합성 문서의 job_id>'
PLAIN_SHA256='<C에서 저장한 평문 SHA256>'
dm_handoff check-job --job "$JOB_ID" --expected-plaintext-sha256 "$PLAIN_SHA256"
```

정상 기대값:

명령/데이터 · json

```
{"state":"COMPLETE","error_code":null,"indexed_chunks":1,"checks":{"job_complete":true,"server_cleanup":true,"host_cleanup":true,"active_versi
on":true,"active_mapping":true,"active_document":true,"parser_ready_clean":true,"es_count_matches":true,"plaintext_matches_fixture":true},"pa
ssed":true}
```

예시의 indexed_chunks: 1은 고정 합격 숫자가 아니다. 실제 수가 1 이상이고 파서 staged count와 같아야 한다. passed: true, 모든 checks가 true, 종료 코드 0이 도구 합격 기준이다. 진행 중/불일치면 종료 코드 2, 설정/실행 오류는 1이다. 도구는 원문·벡터·비밀값을 출력하거나 상태를 고치지 않는다.

도구 결과만으로 디스크의 모든 평문 부재를 증명하지 않는다. 워커와 감시기를 다음 절차로 정상 종료한 뒤, 전용 시험 경로에서 잔존 파일을 별도로 확인한다.

명령/데이터 · powershell

```
New-Item -ItemType File -Path 'D:₩DocMind₩management₩handoff-sample-20261110₩watch.stop'
New-Item -ItemType File -Path 'D:₩DocMind₩management₩handoff-sample-20261110₩ingest.stop'
```

두 프로세스의 정상 종료를 확인한다. 강제 종료하면 아직 정리 중인 작업을 끊을 수 있다. 아래 work_root와 영수증 루트는 **시험 전용** 설정에서 읽는다. 풀은 슬롯별 하위 디렉터리를 사용하므로 재귀적으로 확인한다.

명령/데이터 · powershell

```
$cfg = Get-Content -LiteralPath 'D:₩DocMind₩management₩handoff-sample-20261110₩handoff-worker.private.json' -Raw | ConvertFrom-Json
Get-ChildItem -LiteralPath $cfg.work_root -File -Recurse -Force -ErrorAction Stop | Select-Object FullName,Length
Get-ChildItem -LiteralPath $cfg.cleanup_receipt_root -File -Recurse -Force -ErrorAction Stop | Select-Object FullName,Length
$cipherAfter = (Get-FileHash -LiteralPath $encryptedSample -Algorithm SHA256).Hash
if ($cipherBefore -ne $cipherAfter) { throw '원본 암호문 해시가 변경되었습니다.' }
```

기대: 평문 작업 파일·미처리 receipt 없음, 원본 해시 동일. 폴더가 없거나 접근할 수 없으면 확인 불가로 기록하고 0개로 간주하지 않는다. 슬롯 경로가 설정 루트와 다르면 기존 Get-DocMindWorkerRolePaths로 실제 경로를 확인한다. 시험 결과는 보관하고 운영 색인 삭제 명령을 실행하지 않는다.

UI의 해당 문서 상세에서 확인 표식과 본문을 눈으로 대조한다. 원문 미리보기는 별도 워커가 필요하므로 청크 본문 확인과 구분한다. KURE 검색까지 검증할 때는 외부 연결·인증이 준비된 후 API 명세와 인수인계서의 검색 예제로 별도 기록한다. 이번 색인 합격만으로 검색·답변 전체 합격이라고 적지 않는다.

F. 인수 기록 양식

명령/데이터 · text

```
실행자 / 날짜:
Git SHA / 이미지 ID / 모델 SHA256 일치 여부:
서버 / 프로젝트 / 시험 source ID:
설정: 문서 슬롯 / 파서 / 임베딩 / CPU·메모리 한도:
암호문 SHA256 전후 동일 여부:
평문 fixture SHA256 일치 여부:
job_id / document_id / 활성 청크 수:
check-job 종료 코드 / passed / 실패 checks:
전용 평문·receipt 잔존 파일 수:
UI 확인 표식·본문 일치 여부:
검색 시험: 미실행 또는 별도 결과:
시작·마지막 활성화·정리 완료 시각(계측했을 때만):
미완료 사항 / 담당자:
```

이번 문서 보완에서 도구는 격리 SQLite 시험, AVDC 기존 완료 작업의 읽기 검증, source 등록 dry-run을 수행했다. 위 합성 문서를 새로 암호화·색인하는 종단 간 인수 시험은 수행하지 않았다. 기존 154개 실측과 새 도구 검증 결과를 혼합하지 않는다.

13. 진단·검증 범위·남은 업무

정지·복구 원칙

워커는 실행 때 지정한 stop 파일로 정상 종료를 요청하고 진행 중 작업·정리 ACK가 끝날 때까지 기다린다. 준비된 AVDC 서비스 정지는 sudo /opt/docmind-avdc/manage.sh stop, 상태 확인은 sudo /opt/docmind-avdc/manage.sh ps 다. down -v 또는 볼륨 삭제는 정지 방법이 아니다.

변경 전 실제 Compose·이미지 ID·runtime 소스·모델 식별자를 별도 보관한다. 복구할 때는 해당 서비스의 코드와 설정을 함께 되돌리고 health와 기존 작업·청크를 확인한다. 청킹 버전 복구만으로 이미 만든 청크가 자동으로 이전 결과가 되지는 않는다. DB·색인 전체 삭제를 일반 복구 절차로 사용하지 않는다.

완료·실패 판단과 기본 진단

색인 완료는 작업 등록 수나 UI 문서 수 하나만으로 판단하지 않는다. 성공 작업의 lifecycle_state=COMPLETE, cleanup_state=COMPLETE, host_cleanup_state=COMPLETE, 원본 버전의 활성 상태, 활성 청크 세트 및 ES 항목을 함께 확인한다. 관리 조회는 기존 엔진 서비스/관리 도구를 사용하며 DB 상태를 직접 수정해 완료 처리하지 않는다.

현상·코드	확인할 부분
WORKER_AUTH_FAILED	키 ID·공유키 대응, 시계 차이, nonce·본문 서명. 키 값은 로그에 출력하지 않음
작업이 안 잡힘	source 활성/폴더 매핑, 실제 대기 작업, worker 범위·접근 유효기간·기존 임대
PARSER_BUSY	파서 용량과 클라이언트 동시성. 무제한 즉시 재시도 대신 기존 큐·재시도 경로 사용
PARSER_PDF_NO_SEARCHABLE_TEXT	OCR이 필요한 PDF. 이 구성에서는 정상 색인으로 집계하지 않음
8,192토큰 초과	해당 입력·청킹 경계 점검. 임베딩에서 강제 절단하지 않음
임베딩 503	health, 작업자 생존, 메모리·큐 대기 확인
CLEANUP_FAILED 또는 정리 미완료	평문 잔존·정리 영수증·해당 작업 버전 확인. 무조건 새 작업을 만들지 않음
디스크 부족	실제 볼륨과 이미지/로그 사용량 구분. 엔진 재시작이나 기존 색인 삭제로 덮지 않음

검증된 부분

아래는 기존 기록으로 확인한 범위다. 현재 문서 작성 때문에 시험을 다시 실행하지 않았다.

검증	결과	범위와 한계
실제 154개 새 색인	105,638,237바이트(100.74MiB), 152성공·OCR 필요 2제외, 456쪽·1,596청크	2026-11-10 단회, 기존 색인 0건에서 시작, 서비스 예열 상태
전체 시간	마지막 활성화 184.64초, 전체 작업 종료·호스트 정리 ACK 185.18초	최초 작업 등록부터. 복호화·전송·대기 포함, 준비·백업·사후 검증 제외
서버 전체 CPU	작업 전 평균 4% → 색인 중 평균 25%, 색인 중 관측 최대 83%	약 6초 표본. 순간 최대 전부를 포착한 값이 아님
메모리	DocMind 컨테이너 합계 관측 최대 37.17GiB, 임베딩 33.74GiB	컨테이너 약 1초 표본, 호스트 가용 RAM 최저 18.63GiB

검증	결과	범위와 한계
그룹웨어	HTTP 평균 20.67 → 22.18ms, 색인 중 p95 68.75ms·최대 130.46ms, HTTP 200 31/31	기준 13표본·색인 31표본. 시작 페이지 검사이며 로그인 업무 부하 시험이 아님
원본·정리	원본 154개 SHA256 불변, Windows/API/파서 임시 파일 0, 호스트 정리 완료	해당 시험의 사후 확인
OpenVINO 전환	같은 24입력 CPU 70.159 → 61.697초, PC GPU 39.191초	과거 단회. 현재 154개 GPU 실측은 없음
임베딩 수치	같은 1,359벡터 최대 절대 차이 1.63624e-6	해당 PyTorch/OpenVINO 비교 입력에 한정
검색 회귀	OpenVINO 전환 후 동일 source 12질문의 문서·근거·관련성 상태 12/12 동일	전체 검색 정확도 100%를 뜻하지 않음
청킹 v3	사례집 794 → 440청크, 48토큰 미만 44개·표 108개, 8문항 근거 1위 8/8 유지	단일 사례집 비교. Kordoc 자체는 수정하지 않음

그룹웨어 VM CPU의 정수 지표가 색인 중 0%였다는 것은 실제 CPU 시간이 완전히 0이라는 뜻이 아니다. 서버 전체 CPU와 분모도 다르다. 이번 154개 시험은 KURE 재평가를 병행하지 않았다.

남은 부분 — 미검증·개선 가능성과 운영 인계 구분

항목	현재 판단	이어받을 작업
새 환경 설치 재현	Git 소스·예제는 있음. 최신 AVDC를 빈 서버에서 한 번에 설치하는 완성 패키지는 별도 확인 필요	이미지·모델·상용 복호화 모듈·서버 설정 목록을 수령하고 합성 문서로 재현
source 초기 구성	기존 설치 패키지 초기화 코드와 이번 추가 source 등록 CLI 제공	현재 AVDC에는 새 source만 추가. 실제 원본 매핑·합성 E2E 인수는 담당자가 실행
상시 변경 감지	감시·증분/삭제 코드와 프로토타입 검증 있음	현재 AVDC 조합에서 신규/수정/삭제·폴더 삭제·재시작 E2E 확인. 154개 시험으로 대체 불가
무인 복호화	현재 시험 실행 계정에서 성공	로그아웃·재부팅·서비스 계정 조건은 별도 검증. 운영 담당자와 수행
검색 외부 연결	OpenVINO 검색 회귀 기록은 있음. AVDC 단독 외부 통신 경로는 미완료	승인된 KURE/답변 연결 준비 후 실제 사용자 인증·scope 포함 통합 시험. NAT 보류 유지
OCR·형식 범위	비OCR 문서 경로를 검증	OCR 필요 문서는 별도 처리 정책. 모든 형식·손상·초대형 파일을 검증했다고 보지 않음
파서 수 최적성	현재 2작업자/합산 4개분 한도 사용	22임베딩 작업자 조건의 전역 최적값은 확정하지 않음. 추가 튜닝은 선택 과제
대량·장시간·혼합 부하	154개 단회 및 과거 비교 시험	TB 규모·업무 피크·검색 동시 부하·장시간 장애 복구는 별도. 속도 환산은 보장값이 아님
서버 운영	엔진 인계 범위 밖	자동 기동·모니터링·백업·복구·네트워크·키 관리 담당자 지정

예전 6문서·임베딩 4작업자 시험에서는 파서 1개 81.75초, 2개 83.96초였고 전체 임베딩 작업자의 동시 대기는 10.33초에서 9.08초로만 줄었다. 두 조건 모두 파서 CPU 한도는 4개분이었다. 단회·다른 동시성 조건이므로 파서 2개가 항상 느리다거나 현재 파서 1개가 최적이라고 결론 내리지 않는다.

인수 시 확인 목록

- [ ] 코드 기준 SHA와 실제 적용 코드/이미지/모델 식별자를 함께 받았다. Git 최신 커밋이 서버 전체 이미지와 같다고 추정하지 않는다.

- [ ] 실제 설정·키·모델·복호화 모듈은 승인된 별도 경로로 전달받았다. Git 문서에는 값이 없다.

- [ ] 문서 source 등록과 Windows 루트 매핑, API loopback 전달 경로를 확인했다.

- [ ] 서비스 health와 자원 한도를 확인했다. 기존 PC GPU 경로와 AVDC CPU 경로를 구분했다.

- [ ] 지정된 합성 문서 한 건으로 작업 생성 → 활성 청크 → 정리 ACK까지 재현했다. 이 체크는 인수 담당자가 수행할 항목이며 이번 문서 작성 중 실행한 것은 아니다.

- [ ] OCR 제외·실패 결과를 성공과 구분하며, 검색·상시 동기화·운영 과제의 담당자를 정했다.

이번 보완의 검증

- AVDC의 실제 API Python 환경에서 새 도구의 격리 SQLite/합성 파일 검사 8개 통과: dry-run 무변경, 등록 멱등성, 폴더 순번 충돌 방지, 기존 비활성/다른 프로젝트 보존, 잘못된 ID 거절, 정리·활성 버전 불일치 거절, DOCX XML 및 덮어쓰기 금지.

- AVDC 기존 완료 작업 1건에 새 check-job을 읽기 전용 실행해 활성 청크 10개·ES 개수 일치·모든 상태 검사 통과. 원문·벡터는 출력하지 않았다.

- 실제 DB에는 register-source dry-run만 수행했고 source·폴더·작업 건수가 전후 동일했다. 운영 DB에 새 source를 추가하거나 새 색인을 실행하지 않았다.

- 명령·JSON 예제·문서 링크·Git 추가 파일을 확인했다. 새 설치 전체 재현이나 새 합성 암호문의 종단 간 시험 통과로 확대 해석하지 않는다.

14. 보관·용량·대량 처리 추정과 근거

2026-11-10 확인	현재 상태
문서·색인	154개 등록 / 152개 활성 / 1,596청크. 과거 누적 689개·45,636항목은 백업 후 삭제됨.
디스크	VM 루트 파일시스템 할당 100 GiB. 재색인 후 가용 약 49.6 GiB. 가상 디스크 120 GiB 자체를 더 늘린 것은 아님.
구성·복구	OpenVINO·CPU·메모리 설정과 컨테이너 ID는 측정 전후 동일. 임시 워커 접근은 복원. 원본·설정·키·모델·백업은 Git 밖에 보존.
시스템 구분	AVDC는 CPU/OpenVINO, 여기 PC는 기존 GPU 경로 유지. NAT 변경 없음. Git 반영은 운영 재배포나 새 설치 ZIP 발행이 아님.

1TB는 같은 속도 가정으로 약 20.3일

105,638,237바이트를 정리 ACK까지 185.176초에 처리한 값을 그대로 환산하면 시간당 약 2.05GB, 1TB(10^12바이트)에 약 487시간입니다. 24시간 연속·같은 문서 구성·같은 속도를 가정한 계산이며, TB 규모 실측이나 완료 기한 보장은 아닙니다.

추가로 확인할 범위

반복 편차, 장시간 처리, 업무 피크 시간, 로그인 후 그룹웨어 업무, 검색·색인 동시 부하, 다른 문서 길이 분포와 OCR 대상은 별도 검증이 필요합니다. 100% 허용량이 100% 실제 사용량을 뜻하지 않으며 평균 CPU만으로 추가 증설 효과를 판단하지 않습니다.

근거와 업데이트 위치

[S1] docs/AVDC-INDEXING-154-2026-11-10.md 및 docs/measurements/avdc-fresh154-2026-11-10.json. 개별 문서 정보 없이 같은 실행의 집계·시계열·단계별 평균을 포함합니다.

[S2] docs/AVDC-OPENVINO-DEPLOYMENT-2026-11-06.md. [S3] docs/AVDC-CHUNKING-DEPLOYMENT-2026-11-10.md 및 docs/AVDC-DISK-RECOVERY-2026-11-10.md. 예전 문서의 "현재" 표현은 각 기록 당시 상태입니다.

[S4] https://docs.openvino.ai/nightly/openvino-workflow/running-inference/inference-devices-and-modes/cpu-device.html (2026-11-10 확인). 일반 엔진 기능 설명의 근거이며 원인별 기여도 실측은 아닙니다.

기존 보고서의 내용은 어디에 통합했는가

기존 PDF	통합 위치
10월 15일 GPU 성능·리소스	6장 GPU 과거 기준 및 병렬 시간 해석
11월 3일 98쪽 성능	6장 초기 제약과 실패 기록의 경계
11월 4일·5일·6일 성능 보고서	2·6·7장 같은 입력 비교, 확대·파서·배치·OpenVINO
11월 5일·6일 도입 설명 보고서	3·6·7장 구축 구조, 허용량과 실제 사용량, 원인/한계
11월 6일 종합보고서	2·5·7·14장 GPU, 단계 시간, 이전 154개 결과와 한계
11월 10일 종합보고서	1~5·7·14장 최신 154개, 청킹, 부하·용량
엔진 인수인계서와 보완 5종	8~13장 + 부록: 자산·설정·연동·실행·합격·미완료

중복 설명은 합치고 현재 수치가 바뀐 부분은 최신 기록을 우선했다. 과거 PDF의 페이지 순서나 모든 중간 시도를 그대로 복제한 합본은 아니다. 원본 기록은 삭제하지 않으며 실패·준비·단계 단독 시험은 전체 색인 성능에서 구분한다.

실험 기록 정리 후 기준 루트는 D:₩DocMind₩archive₩avdc-experiments다. 주요 하위 기록: openvino-full-24-r1/comparison.json, stage-comparison.json, scale48-gpu24-r2/result.json, openvino-154-20261106/result.json, cleanup-proof.json, host-load.csv, audit-summary.json, openvino-search-diagnosis-20261106/diagnosis.json, parser2-sdpa/comparison.json. 이전 보고서의 D:₩DocMind-AVDC 경로는 정리 이전 위치다.

GitLab과 GitHub main에 통합본·소스·기존 상세 문서를 함께 보관한다. 실제 키, 원본 문서, 모델 가중치, 상용 복호화 모듈은 Git 외부 자산이다. GitLab 기존 main은 legacy 브랜치에 보존되어 있으며 이 통합 작업은 force push나 실행 설정 변경을 하지 않는다.

부록 A. 실제 확인한 자산 식별자

2026-11-10 읽기 전용 확인 기록. 이미지 ID와 전송 archive SHA256은 서로 다른 식별자다. 아래 JSON에는 실제 비밀 키 값이 없다. Windows 상용 모듈의 재해시는 인수 때 수행한다.

명령/데이터 · json


```
{
  "verified_on": "2026-11-10",
  "method": "pinned SSH read-only Docker inspect and file SHA256",
  "baseline_code": "f5c7bebad51c49d7b4e6a706bfc23ecda29baace",
  "services": {
    "docmind": {
      "image": "sha256:6efabc3403fcd5b37ca6f83ac709fbcb9075cbc7542f4d11ba09db2697189365",
      "configured_image": "docmind-avdc:cpu-noocr-20261102",
      "status": "running",
      "cpu": 0.0,
      "cpuset": "48-55",
      "memory": 2684354560,
      "settings": {
        "DOCMIND_INGESTION_DOCUMENT_CONCURRENCY": "12",
        "MAX_CONCURRENT_EMBEDDING_REQUESTS": "24",
        "KORDOC_CLIENT_CONCURRENCY": "2",
        "PARSER_PLATFORM_KORDOC_CHUNKER_REVISION": "docmind-kordoc-chunker-v3",
        "DOCMIND_SHARED_WORKSPACE_ENABLED": "1",
        "DOCMIND_HOST_WORKER_KEY_ID": "docmind-runtime-host"
      }
    },
    "embedding": {
      "image": "sha256:5b54a0e50412b36e64bf6f8fcc89472d08d11cb203f0fc42fb744b6b576794b6",
      "configured_image": "docmind-gte-openvino:20261106",
      "status": "running",
      "cpu": 88.000004096,
      "cpuset": "0-47,56-95",
      "memory": 55834574848,
      "settings": {}
    },
    "es": {
      "image": "sha256:58a3a280935d830215802322e9a0373faaacdfd646477aa7e718939c2f29292a",
      "configured_image": "elasticsearch:8.11.3",
      "status": "running",
      "cpu": 0.0,
      "cpuset": "48-55",
      "memory": 1610612736,
      "settings": {}
    },
    "minio": {
      "image": "sha256:29a498b24669cae1fed11c1a2fb2b3d73c68829a0a9c0b14e71b386671d38fac",
      "configured_image": "pgsty/silo:RELEASE.2026-08-06T00-00-00Z",
      "status": "running",
      "cpu": 0.0,
      "cpuset": "48-55",
      "memory": 268435456,
      "settings": {}
    },
    "mysql": {
      "image": "sha256:d58ac93387f644e4e040c636b8f50494e78e5afc27ca0a87348b2f577da2b7ff",
      "configured_image": "mysql:8.0.40",
      "status": "running",
      "cpu": 0.0,
```

```
    "cpuset": "48-55",
    "memory": 536870912,
    "settings": {}
  },
  "parser": {
    "image": "sha256:53fc3d95238e2bc0f8dc35676440213b2a36d59a904c7d051bdcf35166475d2d",
    "configured_image": "docmind-kordoc-parser:2026.10.19",
    "status": "running",
    "cpu": 4.0,
    "cpuset": "48-55",
    "memory": 1073741824,
    "settings": {
      "KORDOC_WORKERS": "2"
    }
  },
  "preview": {
    "image": "sha256:9cf6168534fb253c035a46e70f10e15a0e258e65e01311e8fce4045bf2832708",
    "configured_image": "docmind-preview:2026.10.19",
    "status": "running",
    "cpu": 0.0,
    "cpuset": "48-55",
    "memory": 268435456,
    "settings": {}
  },
  "redis": {
    "image": "sha256:640c5e62cea04b6d6f2084232651d0cc70362d31f4f805e7be94dbed6855e8f2",
    "configured_image": "valkey/valkey:8",
    "status": "running",
    "cpu": 0.0,
    "cpuset": "48-55",
    "memory": 134217728,
    "settings": {}
  }
},
"files": {
  "runtime/embedding/service.py": {
    "bytes": 7800,
    "sha256": "778d77a3622f986fe8de6222ba2168081daa7eb13c4c0e9ce50d527df2371f65"
  },
  "runtime/embedding/pool.py": {
    "bytes": 6908,
    "sha256": "6e447a289e6b6d8338361c325291e4d9eaa71df40cd74fba5b10147688592953"
  },
  "runtime/api/office_chunker.py": {
    "bytes": 13505,
    "sha256": "5eccc80e948274aeaaafac0489a7ba22d47bdffa7f0c457cd1034547faac7f13"
  },
  "runtime/api/parser_config.py": {
    "bytes": 13281,
    "sha256": "d6e410e320f2e8d253974e423d4008c7e9062b4bd6dc265c6a81b8b4168f4b77"
  },
  "models/openvino/model.xml": {
    "bytes": 541657,
    "sha256": "0db4102fb18b2724f91e6d404ae651d5ec39a9ae65dd479c7f7ae0e8079e0c19"
  },
  "models/openvino/model.bin": {
    "bytes": 1255093468,
    "sha256": "ca07edf92d30bdfc2fc2b423aea7db6481b94b65163a596fb314b17bc3b3ce17"
  },
  "runtime/api/docmind_ingestion_service.py": {
    "bytes": 74708,
    "sha256": "d85497ea15ee2ba215baa8df31a3102fd5e6e62611ea950fd4c266a982ae6af9"
  },
```

```
    "runtime/api/docmind_api.py": {
      "bytes": 54419,
      "sha256": "0189bab290a295847b80dc0071bc0ed591de287b3a13fe54715260813958fe7e"
    },
    "runtime/api/settings.py": {
      "bytes": 14589,
      "sha256": "0fea0af0d0624038addee3ed9729dd6fea0c97d5e8f760605238a0970534a954"
    },
    "runtime/api/docmind_ingestion_executor.py": {
      "bytes": 1856,
      "sha256": "2d3ce874a4aeffd71d3fcdd3b1421db9d7ddc73b8ab5741bf3d87bc098ea0646"
    },
    "runtime/api/embedding_service.py": {
      "bytes": 9389,
      "sha256": "5daf7e52c83147eee96d068adfe94ad823e99b95d986996de7fb0f429fe1bfb5"
    }
  },
  "model_revision": "9bbca17d9273fd0d03d5725c7a4b0f6b45142062",
  "trusted_code_revision": "40ced75c3017eb27626c9d4ea981bde21a2662f4",
  "note": "Image IDs are not archive file hashes. No full installer archive was created or verified by this handoff."
}
```

부록 B. 통합에 사용한 PDF 원본 식별자

원본 폴더: D:₩DocMind₩reports. 이력 파일은 유지하며 대표 문서는 이 통합본이다. SHA256은 원본 파일 대조용이다.

DocMind_AVDC_98쪽_색인_성능_리소스_보고서_2026-11-03.pdf

SHA256: 5fcbf126f3200c749a8618d723509cc4d1dee61ab3cea832d1a7eeca5d319128

DocMind_AVDC_구축부터_색인완료까지_종합보고서_2026-11-06.pdf

SHA256: 4686a88ab948c05593bdcfab5a913bfeec24365482f89bb55bbcf969d10af60e

DocMind_AVDC_구축부터_색인완료까지_종합보고서_2026-11-10.pdf

SHA256: fd44a74756404ea526b4c5957e5c6fe0e105a373831cb4eca6b2e83304c34e2f

DocMind_AVDC_도입과_최적화_설명보고서_2026-11-05.pdf

SHA256: 0c053df357f09553aad0a2a7191e3a833866d44aad07a0b40927048cf70a4eb0

DocMind_AVDC_도입과_최적화_설명보고서_2026-11-06.pdf

SHA256: cf1d6ee210c6abcacea8e20bc2493010eff7539a176db342e17ce93af2e9a530

DocMind_AVDC_색인_성능_리소스_보고서_2026-11-04.pdf

SHA256: 3aa444c938eb484c251b8961b45d840685c4bcc49c93b762ddc2cd2ff6d3f036

DocMind_AVDC_색인_성능_리소스_보고서_2026-11-05.pdf

SHA256: 97b8a9d87de74c3ba1360cdadbfbc9646aa79bfa17e1b74a8434fcdd0cdea04a

DocMind_AVDC_색인_성능_리소스_보고서_2026-11-06.pdf

SHA256: e771579fee7f5db28afde1731109e4d144db5795a7f7afbe69b7e41a3684efcc

DocMind_인덱싱_성능_리소스_보고서_2026-10-15.pdf

SHA256: bc718608bd0c874e59418a98d6aae2f80c393a2fa9273e24bb4864bb182cf7c2